

- [Key components](#)
 - [NIM microservices](#)
- [Getting started](#)
 - [Install system requirements](#)
 - [Obtain API keys](#)
 - [Set up the workflow repository](#)
 - [Set up the environment file](#)
 - [Authenticate Docker with NGC](#)
 - [Build the Docker containers](#)
 - [Start the Docker containers](#)
- [Running the Workflow](#)
 - [From the quick start user guide notebook](#)
 - [From the command line](#)
 - [Command line interface \(CLI\) reference](#)
 - [Configuration file reference](#)
 - [NGINX caching server](#)
- [Customizing the Workflow](#)
 - [Customizing the SBOM input](#)
 - [Customizing the LLM models](#)
 - [Customizing the embedding model](#)
 - [Customizing the Output](#)
 - [Adding Test Time Compute \(TTC\)](#)
- [Evaluating the Workflow](#)
 - [Running Evaluation \(Overview\)](#)
 - [Inputting Evaluation Datasets](#)
 - [Running Multiple Evaluation Repetitions](#)
 - [Accuracy and Consistency Evaluators](#)
 - [Writing Custom Evaluators](#)
- [Troubleshooting](#)
 - [Git LFS issues](#)
 - [Container build issues](#)
 - [NGINX caching server](#)
 - [Service errors](#)
 - [macOS Workarounds](#)
- [Testing and validation](#)
- [Roadmap](#)
- [Cite](#)
- [License](#)
- [Terms of Use](#)

Overview

This repository is what powers the [build experience](#), showcasing vulnerability analysis for container security using NVIDIA NIM microservices and [NVIDIA NeMo Agent Toolkit](#).

The NVIDIA AI Blueprint demonstrates accelerated analysis on common vulnerabilities and exposures (CVE) at an enterprise scale, reducing mitigation from days and hours to just seconds. While traditional methods require substantial manual effort to pinpoint solutions for vulnerabilities, these technologies enable quick, automatic, and actionable CVE risk analysis using large language models (LLMs) and retrieval-augmented generation (RAG). With this blueprint, security analysts can expedite the process of determining whether a software package includes exploitable and vulnerable components using LLMs and event-driven RAG triggered by the creation of a new software package or the detection of a CVE.

Software components

The following are used by this blueprint:

- [NIM of meta/llama-3.1-70b-instruct](#)
- [NIM of nvidia/nv-embedqa-e5-v5](#)
- [NVIDIA NeMo Agent Toolkit](#)

Target audience

This blueprint is for:

- Security analysts and IT engineers:** People analyzing vulnerabilities and ensuring the security of containerized environments.
- AI practitioners in cybersecurity:** People applying AI to enhance cybersecurity, particularly those interested in using the NeMo Agent Toolkit and NIMs for faster vulnerability detection and analysis.

Prerequisites

- NVAIE developer licence
- API keys for vulnerability databases, search engines, and LLM model service(s).
 - Details can be found in this later section: [Obtain API keys](#)

Hardware requirements

Below are the hardware requirements for each component of the vulnerability analysis workflow.

The overall hardware requirements depend on selected workflow configuration. At a minimum, the hardware requirements for workflow operation must be met. The LLM NIM and Embedding NIM hardware requirements only need to be met if self-hosting these components. See [Using self-hosted NIMs](#), [Customizing the LLM models](#) and [Customizing the embedding model](#) sections for more information.

- **(Optional) LLM NIM:** [Meta Llama 3.1 70B Instruct Support Matrix](#)
 - This workflow makes heavy use of parallel LLM calls to accelerate processing. For improved parallel performance (for example, in production workloads), we recommend 8x or more H100s for LLM inference.
- **(Optional) Embedding NIM:** [NV-EmbedQA-E5-v5 Support Matrix](#)

Operating system requirements

Officially Supported: Ubuntu and other Linux distributions

Limited Support for macOS: Testing and development of the workflow may be possible on macOS, however macOS is not officially supported or tested for this blueprint. Platform differences may require extra troubleshooting or impact performance. See [macOS Workarounds](#) for known issues and workarounds. In addition, self-hosting NIMs is not supported on macOS (requires NVIDIA GPUs not available on Mac hardware). For production deployments, use Linux-based systems.

API definition

[OpenAPI Specification](#)

Use case description

Determining the impact of a documented CVE on a specific project or container is a labor-intensive and manual task, especially as the rate of new reports into the [CVE database](#) accelerates. This process involves the collection, comprehension, and synthesis of various pieces of information to ascertain whether immediate remediation is necessary upon the identification of a new CVE.

Current challenges in CVE analysis:

- **Information collection:** The process involves significant manual labor to collect and synthesize relevant information.
- **Decision complexity:** Decisions on whether to update a library impacted by a CVE often hinge on various considerations, including:
 - **Scan false positives:** Occasionally, vulnerability scans may incorrectly flag a library as vulnerable, leading to a false alarm.
 - **Mitigating factors:** In some cases, existing safeguards within the environment may reduce or negate the risk posed by a CVE.
 - **Lack of required environments or dependencies:** For an exploit to succeed, specific conditions must be met. The absence of these necessary elements can render a vulnerability irrelevant.
- **Manual documentation:** Once an analyst has determined the library is not affected, a Vulnerability Exploitability eXchange (VEX) document must be created to standardize and distribute the results.

The efficiency of this process can be significantly enhanced through the deployment of an automated LLM agent workflow, leveraging generative AI to improve vulnerability defense while decreasing the load on security teams.

How it works

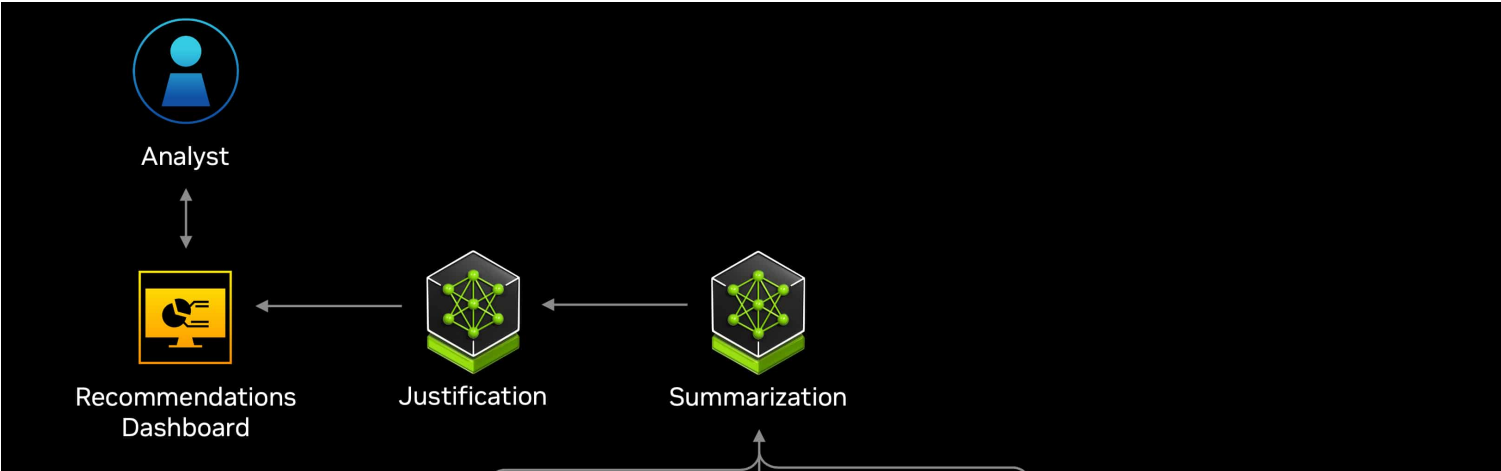
The workflow operates using a Plan-and-Execute-style LLM pipeline for CVE impact analysis. The process begins with an LLM planner that generates a context-sensitive task checklist. This checklist is then executed by an LLM agent equipped with Retrieval-Augmented Generation (RAG) capabilities. The gathered information and the agent's findings are subsequently summarized and categorized by additional LLM nodes to provide a final verdict.

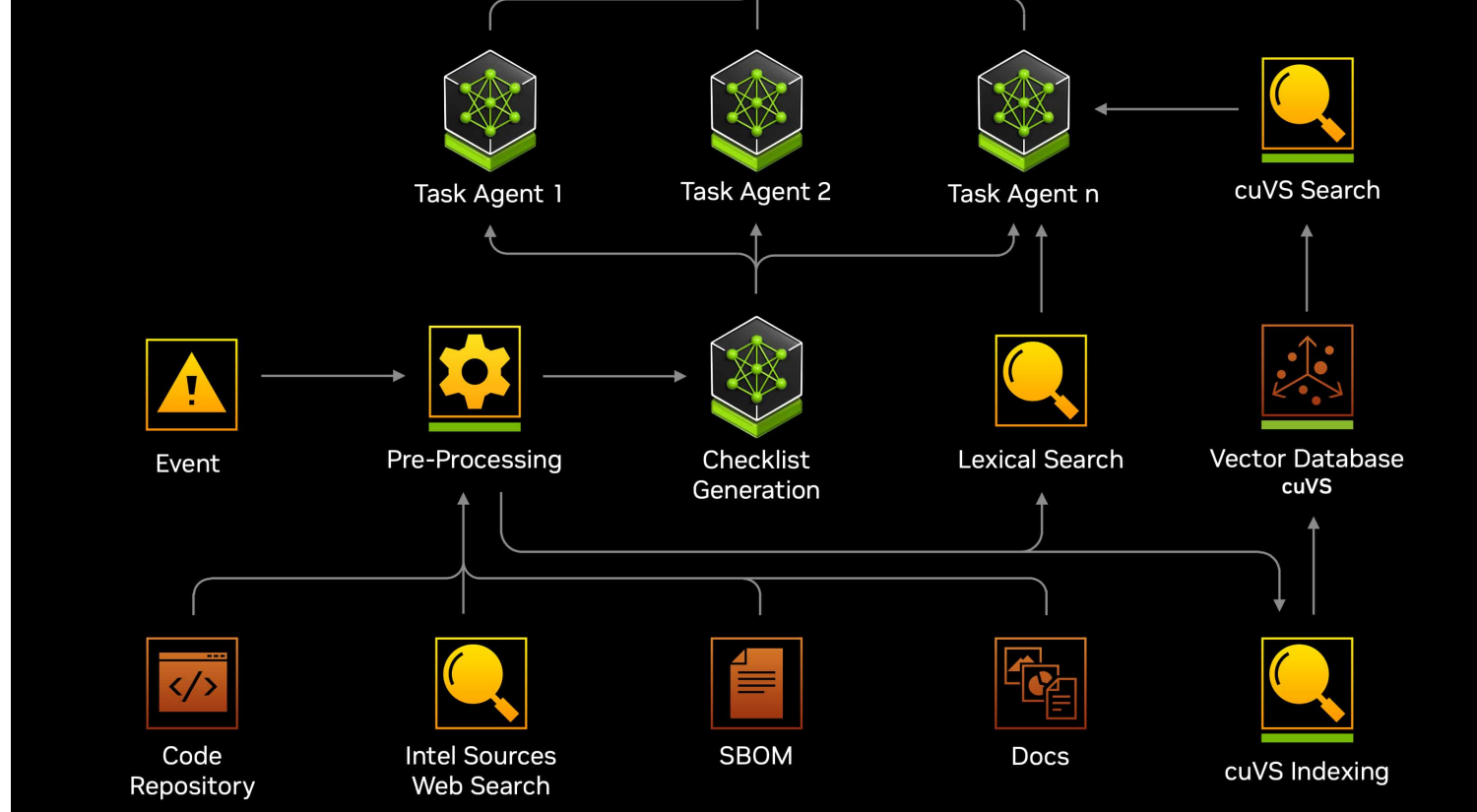
 Tip

The workflow is adaptable, with support for NIM and OpenAI LLM APIs. NIM models can be hosted on [build.nvidia.com](#) or self-hosted.

Key components

The detailed architecture consists of the following components:





- **Security scan result:** The workflow begins by inputting the identified CVEs from a container security scan as input. This can be generated from a container image scanner of your choosing such as [Anchore](#).
- **PreProcessing:** All the below actions are encapsulated by multiple NeMo Agent toolkit functions to prepare the data for use with the LLM engine.
 - **Code repository and documentation:** The blueprint pulls code repositories and documentation provided by the user. These repositories are processed through an embedding model, and the resulting embeddings are stored in vector databases (VDBs) for the agent's reference.
 - **Vector database:** Various vector databases can be used for the embedding. We currently utilize FAISS for the VDB because it does not require an external service and is simple to use. Any vector store can be used, such as NVIDIA cuVS, which would provide accelerated indexing and search.
 - **Lexical search:** As an alternative, a lexical search is available for use cases where creating an embedding is impractical due to a large number of source files in the target container.
 - **Software Bill of Materials (SBOM):** A Software Bill of Materials (SBOM) is a machine-readable manifest of all the dependencies of a software package or container. The blueprint cross-references every entry in the SBOM for known vulnerabilities and looks at the code implementation to see whether the implementation puts users at risk—just as a security analyst would do. For this reason, starting with an accurate SBOM is an important first step. SBOMs can be generated for any container using the open-source tool [Syft](#). For more information on generating SBOMs for your containers, see the [SBOM documentation](#).
 - **Web vulnerability intel:** The system collects detailed information about each CVE through web scraping and data retrieval from various public security databases, including GHSA, Redhat, Ubuntu, and NIST CVE records, as well as tailored threat intelligence feeds.
- **Core LLM engine:** The below actions comprise the core LLM engine and are each implemented as NeMo Agent toolkit functions within the workflow.
 - **Checklist generation:** Leveraging the gathered information about each vulnerability, the checklist generation node creates a tailored, context-sensitive task checklist designed to guide the impact analysis. (See [src/vuln_analysis/functions/cve_checklist.py](#).)
 - **Task agent:** At the core of the process is an LLM agent iterating through each item in the checklist. For each item, the agent answers the question using a set of tools which provide information about the target container. The tools tap into various data sources (web intel, vector DB, search etc.), retrieving relevant information to address each checklist item. The loop continues until the agent resolves each checklist item satisfactorily. (See [src/vuln_analysis/functions/cve_agent.py](#).)
 - **Summarization:** Once the agent has compiled findings for each checklist item, these results are condensed by the summarization node into a concise, human-readable paragraph. (See [src/vuln_analysis/functions/cve_summarize.py](#).)
 - **Justification Assignment:** Given the summary, the justification status categorization node then assigns a resulting VEX (Vulnerability Exploitability eXchange) status to the CVE. We provided a set of predefined categories for the model to choose from. (See [src/vuln_analysis/functions/cve_justify.py](#).) If the CVE is deemed exploitable, the reasoning category is "vulnerable." If there is no vulnerable packages detected from the SBOM or insufficient intel gathered the agent is bypassed and an appropriate label is provided. If it is not exploitable, there are 10 different reasoning categories to explain why the vulnerability is not exploitable in the given environment:
 - `false_positive`
 - `code_not_present`
 - `code_not_reachable`
 - `requires_configuration`
 - `requires_dependency`
 - `requires_environment`
 - `protected_by_compiler`
 - `protected_at_runtime`
 - `protected_by_perimeter`
 - `protected_by_mitigating_control`
- **Output:** At the end of the workflow run, an output file including all the gathered and generated information is prepared for security analysts for a final review. (See [src/vuln_analysis/functions/cve_file_output.py](#).)

 Warning

All output should be vetted by a security analyst before being used in a cybersecurity application.

NIM microservices

The NeMo Agent toolkit can utilize various embedding model and LLM endpoints, and is optimized to use [NVIDIA NIM microservices](#) (NIMs). NIMs are pre-built containers for the latest AI models that provide industry-standard APIs and optimized inference for the given model and hardware. Using NIMs enables easy deployment and scaling for self-hosted model inference.

The current default embedding NIM model is `nv-embedqa-e5-v5`, which was selected to balance speed and overall workflow accuracy. The current default LLM model is the `llama-3.1-70b-instruct` NIM, with specifically tailored prompt engineering and edge case handling. Other models are able to be substituted for either the embedding or LLM model, such as smaller, fine-tuned NIM LLM models or other external LLM inference services. Subsequent updates will provide more details about fine-tuning and data flywheel techniques.

 Note

Within the NeMo Agent toolkit workflow, the LangChain framework is employed to deploy all LLMs and agents, and the LangGraph framework is used for orchestration, streamlining efficiency and reducing the need for duplicative efforts.

 Tip

Routinely checked validation datasets are critical to ensuring proper and consistent outputs. Learn more about our test-driven development approach in the section on [testing and validation](#).

Getting started

 Tip

Please refer to the [Troubleshooting](#) section for common errors encountered during setup.

Install system requirements

- [git](#)
- [git-lfs](#)
- Since the workflow uses [NVIDIA NeMo Agent Toolkit](#), the [NeMo Agent toolkit requirements](#) also need to be installed.

Obtain API keys

To run the workflow you need to obtain API keys for the following APIs. These will be needed in a later step to [Set up the environment file](#).

- **Required API Keys:** These APIs are required by the workflow to retrieve vulnerability information from databases, perform online searches, and execute LLM queries.
 - GitHub Security Advisory (GHSa) Database
 - Follow [these](#) instructions to create a personal access token. No repository access or permissions are required for this API.
 - This will be used in the `GHSa_API_KEY` environment variable.
 - National Vulnerability Database (NVD)
 - Follow [these](#) instructions to create an API key.
 - This will be used in the `NVD_API_KEY` environment variable.
 - SerpApi
 - Go to <https://serpapi.com/> and create a SerpApi account. Once signed in, navigate to Your Account > Api Key.
 - This will be used in the `SERPAPI_API_KEY` environment variable.
 - NVIDIA Inference Microservices (NIM)
 - There are two possible methods to generate an API key for NIM:
 - Sign in to the [NVIDIA Build](#) portal with your email.
 - Click on any model that displays the "Run Anywhere" tag, such as [this one](#).
 - Click "Deploy" in the top menu, then click "Get API Key", and finally, click "Generate Key".
 - Sign in to the [NVIDIA NGC](#) portal with your email.
 - After logging in, click on your user at the top right to open a dropdown menu. Click on the first menu item to select your organization. You must select an organization which has NVIDIA AI Enterprise (NVAIE) enabled.
 - In the same dropdown menu as the previous step, select "Setup."
 - Click the "Generate API Key" option and then the "+ Generate Personal Key" button to create your API key.
 - This will be used in the `NVIDIA_API_KEY` environment variable.

The workflow can be configured to use other LLM services as well, see the [Customizing the LLM models](#) section for more info.

Set up the workflow repository

Clone the repository and set an environment variable for the path to the repository root.

```
export REPO_ROOT=$(git rev-parse --show-toplevel)
```



All commands are run from the repository root unless otherwise specified.

Set up the environment file

First we need to create an `.env` file in the `REPO_ROOT` , and add the API keys you created in the earlier [Obtain API keys](#) step.

```
cd $REPO_ROOT
cat <<EOF > .env
GHSA_API_KEY="your GitHub personal access token"
NVD_API_KEY="your National Vulnerability Database API key"
NVIDIA_API_KEY="your NVIDIA Inference Microservices API key"
SERPAPI_API_KEY="your SerpApi API key"
EOF
```

These variables need to be exported to the environment:

```
export $(cat .env | xargs)
```

Authenticate Docker with NGC

In order to pull images required by the workflow from NGC, you must first authenticate Docker with NGC. You can use same the NVIDIA API Key obtained in the [Obtain API keys](#) section (saved as `NVIDIA_API_KEY` in the `.env` file).

```
echo "${NVIDIA_API_KEY}" | docker login nvcr.io -u '$oauthtoken' --password-stdin
```

Build the Docker containers

Next, build the `vuln-analysis` container from source using the following command. This ensures that the container includes all the latest changes from the repository.

```
cd $REPO_ROOT

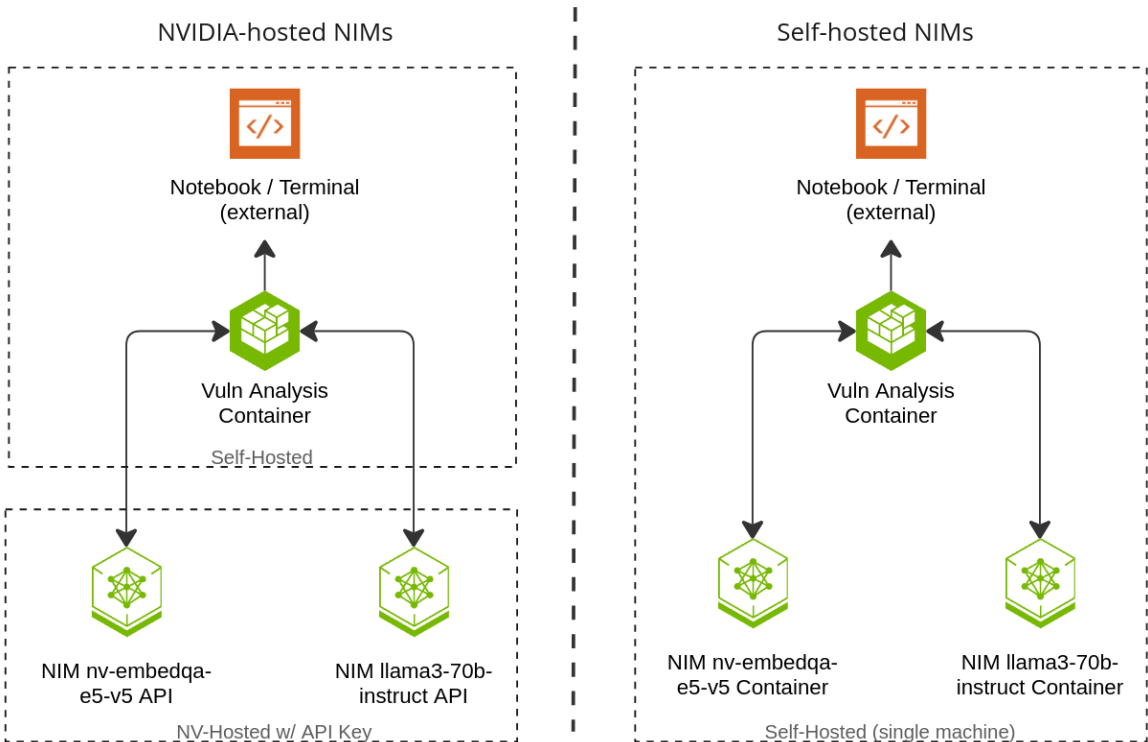
# Build the vuln-analysis container
docker compose build vuln-analysis
```

Start the Docker containers

There are two supported configurations for starting the Docker containers. Both configurations utilize `docker compose` to start the service:

- [NVIDIA-hosted NIMs](#):** The workflow is run with all computation being performed by NIMs hosted in NVIDIA GPU Cloud. This is the default configuration and is **recommended** for most users getting started with the workflow.
 - When using NVIDIA-hosted NIMs, only the `docker-compose.yml` configuration file is required.
- [Self-hosted NIMs](#):** The workflow is run using self-hosted LLM NIM services. This configuration is more advanced and requires additional setup to run the NIM services locally.
 - When using self-hosted NIMs, both the `docker-compose.yml` and `docker-compose.nim.yml` configuration files are required.

These two configurations are illustrated by the following diagram:



Before beginning, ensure that the environment variables are set correctly. Both configurations require the same environment variables to be set. More information on setting these variables can be found in the [Obtain API keys](#) section.

 **Tip**

The container binds to port 8080 by default. If you encounter a port collision error (for example, `Bind for 0.0.0.0:8080 failed: port is already allocated`), you can set the environment variable `NGINX_HOST_HTTP_PORT` to specify a custom port before launching `docker compose` . For example:

```
export NGINX_HOST_HTTP_PORT=8081

#... docker compose commands...
```

Using NVIDIA-hosted NIMs

When running the workflow in this configuration, only the `vuln-analysis` service needs to be started since we will utilize NIMs hosted by NVIDIA. The `vuln-analysis` container can be started using the following command:

```
cd ${REPO_ROOT}
docker compose up -d
```

The command above starts the container in the background using the detached mode, `-d`. We can confirm the container is running via the following command:

```
docker compose ps
```

Next, we need to attach to the `vuln-analysis` container to access the environment where the workflow [command line tool](#) and dependencies are installed.

```
docker compose exec -it vuln-analysis bash
```

Continue to the [Running the workflow](#) section to run the workflow.

Using self-hosted NIMs

To run the workflow using self-hosted NIMs, we use a second `docker compose` configuration file, `docker-compose.nim.yml`, which adds the self-hosted NIM services to the workflow. Utilizing a second configuration file allows for easy switching between the two configurations while keeping the base configuration file the same.

 Note

The self-hosted NIM services require additional GPU resources to run. With this configuration, the LLM NIM, embedding model NIM, and the `vuln-analysis` service will all be launched on the same machine. Ensure that you have the necessary hardware requirements for all three services before proceeding (multiple services can share the same GPU).

To use multiple configuration files, we need to specify both configuration files when running the `docker compose` command. You will need to specify both configuration files for every `docker compose` command. For example:

```
docker compose -f docker-compose.yml -f docker-compose.nim.yml [NORMAL DOCKER COMPOSE COMMAND]
```

For example, to start the `vuln-analysis` service with the self-hosted NIMs, you would run:

```
cd ${REPO_ROOT}
docker compose -f docker-compose.yml -f docker-compose.nim.yml up -d
```

Next, we need to attach to the `vuln-analysis` container to access the environment where the workflow [command line tool](#) and dependencies are installed.

```
docker compose -f docker-compose.yml -f docker-compose.nim.yml exec -it vuln-analysis bash
```

Continue to the [Running the workflow](#) section to run the workflow.

Running the Workflow

 Tip

Please refer to the [Troubleshooting](#) section for common errors encountered while running the workflow.

Once the services have been started, the workflow can be run using either the [Quick start user guide notebook](#) for an interactive step-by-step process, or directly from the [command line](#).

From the quick start user guide notebook

To run the workflow in an interactive notebook, connect to the Jupyter notebook at <http://localhost:8000/lab>. Once connected, navigate to the notebook located at [quick_start/quick_start_guide.ipynb](#) and follow the instructions.

 Tip

If you are running the workflow on a remote machine, you can forward the port to your local machine using SSH. For example, to forward port 8000 from the remote machine to your local machine, you can run the following command from your local machine:

```
ssh -L 8000:127.0.0.1:8000 <remote_host_name>
```

From the command line

The vulnerability analysis workflow is designed to be run using the `python` command line tool installed within the `vuln-analysis` container.

The vulnerability analysis workflow is designed to be run using the `nat` command line tool installed within the `vuln-analysis` container. This section describes how get started using the command line tool. For more detailed information about the command line interface, see the [NeMo Agent toolkit Command Line Interface \(CLI\)](#) documentation.

Workflow configuration file

The workflow settings are controlled using configuration files. These are YAML files that define the functions, tools, and models to use in the workflow. Example configuration files are located in the `configs/` folder.

 Note

The `configs/` and `data/` directories are symlinks pointing to the actual file locations in the `src/vuln_analysis/configs/` and `src/vuln_analysis/data/` directories respectively. The symlinks are available for convenience.

A brief description of each configuration file is as follows:

- `config.yml` : This configuration file defines the functions, tools, and models used by the vulnerability analysis workflow, as described above in [Key Components](#) section.
- `config-tracing.yml` : This configuration file is identical to `config.yml` but adds configuration for observing traces of this workflow in Phoenix.

There are three main modalities that the workflow can be run in using the following commands:

- `nat run` : The workflow will process the input data, then it will shut down after it is completed. This modality is suitable for rapid iteration during testing and development.
- `nat serve` : The workflow is turned into a microservice which will run indefinitely, which is suitable for using in production.
- `nat eval` : Similar to the `nat run` command. However, in addition to running the workflow, it is also used for profiling and evaluating accuracy of the workflow.

For a breakdown of the configuration file and available options, see the [Configuration file reference](#) section. To customize the configuration files for your use case, see [Customizing the workflow](#).

Example command: `nat run`

The workflow can be started using the following command:

```
nat run --config_file=${CONFIG_FILE} --input_file=data/input_messages/morpheus:23.11-runtime.json
```

In the command, `${CONFIG_FILE}` is the path to the configuration file you want to use. For example, to run the workflow with the `config.yml` configuration file, you would run:

```
nat run --config_file=configs/config.yml --input_file=data/input_messages/morpheus:23.11-runtime.json
```

When the workflow runs to completion, you should see logs similar to the following:

```
Vulnerability 'GHSA-3f63-hfp8-52jq' affected status: FALSE. Label: code_not_reachable
Vulnerability 'CVE-2023-50782' affected status: FALSE. Label: requires_configuration
Vulnerability 'CVE-2023-36632' affected status: FALSE. Label: code_not_present
Vulnerability 'CVE-2023-43804' affected status: TRUE. Label: vulnerable
Vulnerability 'GHSA-cxfr-5q3r-2rc2' affected status: TRUE. Label: vulnerable
Vulnerability 'GHSA-554w-xh4j-8w64' affected status: TRUE. Label: vulnerable
Vulnerability 'GHSA-3ww4-gg4f-jr7f' affected status: FALSE. Label: requires_configuration
Vulnerability 'CVE-2023-31147' affected status: FALSE. Label: code_not_present
-----
Workflow Result:
{"input":{"scan":{"id":"8351fd75-4798-42c9-81d8-a43d7df838fd","type":null,"started_at":"2025-06-25T20:21:19.69825:
...
```

 Warning

The output you receive from the workflow may not be identical as the output in the example above. The output may vary due to the non-deterministic nature of the LLM models.

Reviewing the workflow output

The workflow generates two types of output:

1. JSON output (`.tmp/output.json`): A single machine-readable JSON file containing comprehensive workflow execution details for all analyzed CVEs. This file is designed to capture all workflow input, intermediate information, and output for auditing and downstream processing in production systems. The output JSON includes the following top level fields:

- `input` : contains the inputs that were provided to the workflow, such as the container and repo source information, the list of vulnerabilities to scan, etc.
- `info` : contains additional information collected by the workflow for decision making. This includes paths to the generated VDB files, intelligence from various vulnerability databases, the list of SBOM packages, and any vulnerable dependencies that were identified.
- `output` : contains the output from the core LLM Engine, including the generated checklist, analysis summary, and justification assignment.

2. Markdown reports (`.tmp/vulnerability_markdown_reports/`): Individual human-readable reports formatted for easy review by security analysts.

Directory structure:

```
.tmp/vulnerability_markdown_reports/
└─ vulnerability_reports_<container_name>/
    ├── CVE-YYYY-XXXXX.md
    ├── CVE-YYYY-YYYYY.md
    └─ ...
```

A separate folder is created for each container in the input message. Each folder contains a separate markdown file for each CVE affecting the container. For detailed investigation of specific CVEs, navigate to the corresponding markdown file in

```
.tmp/vulnerability_markdown_reports/vulnerability_reports_<container_name>/
```

Report sections:

Each markdown report contains:

- **Header:** Container name, SBOM info, and overall status (Exploitable/Not Exploitable)
- **CVE Description:** Full vulnerability description with GHSA ID and dispute status if applicable
- **Severity and CVSS Score:** Table showing severity ratings and CVSS scores from multiple sources (GHSA, NVD, RHSA, Ubuntu)
- **EPSS Score:** Exploit Prediction Scoring System probability and percentile
- **Summary:** High-level analysis conclusion with key findings
- **Justification:** VEX status label (e.g., `vulnerable`, `code_not_present`, `code_not_reachable`) with reasoning
- **Vulnerable SBOM Dependencies:** Table of packages from the SBOM that match the CVE
- **Checklist:** Context-sensitive questions generated by the LLM to guide the investigation
- **Checklist Details:** Step-by-step breakdown showing each tool call (Code QA, Documentation QA, Internet Search), inputs, outputs, and source documents retrieved
- **References:** Links to authoritative sources for the CVE

 **Tip**

To return detailed steps taken by the LLM agent in the output, set `return_intermediate_steps` to `true` for the `cve_agent_executor` function in the configuration file. This can be helpful for explaining the output, and for troubleshooting unexpected results.

Reviewing an example vulnerability report

An example Markdown-formatted report can be found [here](#). The report opens with key information about the analysis context and gathered CVE intelligence for a quick summary view.

Vulnerability Analysis Report for CVE-2023-36632

Container Analyzed: `nvcr.io/nvidia/morpheus/morpheus:23.11-runtime`

SBOM Info: `type='file' file_path='data/sboms/nvcr.io/nvidia/morpheus/morpheus:v23.11.01-runtime.sbom'`

Status: Exploitable

CVE-2023-36632

GHSA-ID: GHSA-gv66-v8c8-v69c

Disputed [source](#)

The legacy `email.utils.parseaddr` function in Python through 3.11.4 allows attackers to trigger "RecursionError: maximum recursion depth exceeded while calling a Python object" via a crafted argument. This argument is plausibly an untrusted value from an application's input data that was supposed to contain a name and an e-mail address. NOTE: `email.utils.parseaddr` is categorized as a Legacy API in the documentation of the Python email package. Applications should instead use the `email.parser.BytesParser` or `email.parser.Parser` class. [references](#)

Source	Severity	CVSS Score	Vector	Published At	Last Updated
GHSA	UNKNOWN	N/A	N/A	2023-06-25	2023-11-06
NVD	HIGH	7.5	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H	2023-06-25	2024-11-21
RHSA	N/A	N/A	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:N	2023-06-25	N/A
Ubuntu	HIGH	7.5	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H	2023-06-25	2024-07-24

EPSS Score

EPSS	Percentile
0.00121	0.47904

The rest of the report provides the final summary and justification, followed by step-by-step details of the agent's investigation process, including tool calls and retrieved source documents. This "under-the-hood" view allows analysts to explore the logic and rationale behind the reported status:

Vulnerability Analysis

Summary (Exploitable)

Based on the provided Checklist and Findings, the CVE is exploitable. Specifically, the application does not handle exceptions like `RecursionError` when processing input data (Checklist Item 1), and the Docker container's error handling mechanisms are not explicitly designed to handle `RecursionError` exceptions (Checklist Item 2). Additionally, there is a potential source of uncontrolled recursion in the `ProducerConsumerQueue` class due to the use of a recursive lock (Checklist Item 3). These findings suggest that the CVE is exploitable, as the application and Docker container do not have adequate measures in place to prevent or handle `RecursionError` exceptions.

Justification

label: vulnerable

The analysis concludes that the CVE is exploitable due to the application's and Docker container's inadequate handling of `RecursionError` exceptions and the presence of a potential source of uncontrolled recursion, indicating that the package is vulnerable to the CVE and needs to be patched.

Vulnerable SBOM Dependencies

SBOM Name	SBOM Version	Dependency Name	Dependency Version	Relation	System
python	3.10.12	python	3.10.12	SELF	binary
python	3.10.13	python	3.10.13	SELF	binary

Checklist

1. Assess Input Data Handling
1. Docker Container Code QA System

2. Docker Container Code QA System

3. Docker Container Developer Guide QA System
2. Review Error Handling Mechanisms
1. Docker Container Code QA System

2. Docker Container Developer Guide QA System

3. Internet Search
3. Inspect Code for Uncontrolled Recursion
1. Docker Container Code QA System

2. Docker Container Code QA System

Example command: nat serve

Similarly, to run the workflow with the nat serve command, you would run:

```
nat serve --config_file=configs/config.yml --host 0.0.0.0 --port 26466
```

This command starts an HTTP server that listens on port 26466 and runs the workflow indefinitely, waiting for incoming data to process. This is useful if you want to trigger the workflow on demand via HTTP requests.

Once the server is running, you can send a POST request to the /generate endpoint with the input parameters in the request body. The workflow will process the input data and return the output in the terminal and the given output path in the config file.

Here's an example using curl to send a POST request. From a new terminal outside of the container, go to the root of the cloned git repository, and run:

```
curl -X POST --url http://localhost:26466/generate --header 'Content-Type: application/json' --data @data/input_messages/runtime.json
```

In this command:

- http://localhost:26466/generate is the URL of the server and endpoint.
- The -d option specifies the data file being sent in the request body. In this case, it's pointing to the input file morpheus:23.11-runtime.json under the data/input_messages/ directory. You can refer to this file as an example of the expected data format.
 - Since it uses a relative path, it's important to run the curl command from the root of the git repository. Alternatively, you can modify the relative path in the command to directly reference the example json file.

After processing the request, the server will return the the results from the curl request and save the results to the output path specified in the configuration file. The server will also display log and summary results from the workflow as it's running. Additional submissions to the server will append the results to the specified output file.

The workflow also provides a /generate/async endpoint for sending requests to workflow asynchronously. The following curl command is an example of using this endpoint. The request is processed the same as the /generate endpoint but without blocking until the request has completed processing.

```
curl -X POST --url http://localhost:26466/generate/async --header 'Content-Type: application/json' --data @data/input_messages/runtime.json
```

The above command immediately returns a response similar to the following:

```
{"job_id": "f79951ee-aeaa-4a2d-b103-a0a8c15e314b", "status": "submitted"}
```

The status of the request can then be polled using /generate/async/job/{job_id} . For example,

```
curl --request GET --url http://localhost:26466/generate/async/job/f79951ee-aeaa-4a2d-b103-a0a8c15e314b
```

Note

When using the /generate/async endpoint, you can ensure that standard output and errors are visible in the server logs by using the -dask_workers=threads flag with the nat serve command:

```
nat serve --config_file=configs/config.yml --host 0.0.0.0 --port 26466 --dask_workers=threads
```

This flag configures the Dask scheduler to use threaded workers instead of process-based workers, allowing print statements and logging from your workflow to appear directly in the server logs.

When processing multiple async requests concurrently, logs from different requests can be interleaved in the server logs, making it difficult to trace individual requests. If you need to ensure clean, sequential logs for debugging purposes, you can limit the server to process one async request at a time:

```
nat serve --config_file=configs/config.yml --host 0.0.0.0 --port 26466 --dask_workers=threads --workers=1 --max_concurrent_async_requests=1
```

Note that this configuration will reduce throughput as the service will only process one async request at a time.

For more information about available flags for the `nat serve` command, refer to the [NeMo Agent Toolkit API Server documentation](#).

Example command: `nat eval`

The workflow configuration file, [config.yml](#), includes settings in the `eval` section for use with the [NeMo Agent Toolkit Profiler](#) feature, which can be used to analyze and optimize workflow execution. The profiler can be started using the following command:

```
nat eval --config_file=configs/config.yml
```

Reviewing the profiling results

The profiler collects usage statistics and stores them in the `.tmp/eval/cve_agent` directory as configured in [config.yml](#). In this directory, you will find the following files:

- all_requests_profiler_traces.json
- gantt_chart.png
- inference_optimization.json
- standardized_data_all.csv
- workflow_output.json
- workflow_profiling_metrics.json
- workflow_profiling_report.txt

 Tip

To analyze these statistics, we highly recommend referring to the NeMo Agent Toolkit documentation, which provides a [comprehensive guide for analyzing profiling results](#) with detailed visualizations and step-by-step examples, including:

- Prompt vs Completion Token Analysis:** Scatter plots to compare token usage across LLMs
- Workflow Runtime Analysis:** Box plots showing runtime distributions on log scale
- Token Efficiency Metrics:** Bar charts comparing total token usage per model
- Bottleneck Analysis:** Gantt charts showing where models spend time during execution
- Step-by-step walkthrough:** Complete example of profiling a workflow and interpreting results

Observing traces in Phoenix

A separate workflow configuration file, [config-tracing.yml](#) is provided that enables tracing in the workflow using [Phoenix](#).

First, run the following command in separate terminal within the container to start a Phoenix server on port 6006:

```
phoenix serve
```

You can now use `config-tracing.yml` to run the workflow with tracing enabled:

```
nat run --config_file=configs/config-tracing.yml --input_file=data/input_messages/morpheus:23.11-runtime.json
```

Tracing will also be enabled with `config-tracing.yml` when running workflow with `nat serve` and `nat eval`.

Open your browser and navigate to <http://localhost:6006> to view the traces.

Command line interface (CLI) reference

The main entrypoint is the `nat` CLI tool with built-in documentation using the `--help` command. For example, to see what commands are available, you can run:

```
(.venv) root@04a3a12a687d:/workspace# nat --help
Usage: nat [OPTIONS] COMMAND [ARGS]...

Main entrypoint for the NeMo Agent Toolkit CLI

Options:
  --version            Show the version and exit.
  --log-level [debug|info|warning|error|critical]
                        Set the logging level [default: INFO]
  --help              Show this message and exit.

Commands:
  configure  Configure NeMo Agent Toolkit developer preferences.
  eval       Evaluate a workflow with the specified dataset.
  info       Provide information about the local NeMo Agent Toolkit environment.
  mcp        Run a NeMo Agent Toolkit workflow using the mcp front end.
  registry   Utility to configure NeMo Agent Toolkit remote registry channels.
  run        Run a NeMo Agent Toolkit workflow using the console front end.
  serve      Run a NeMo Agent Toolkit workflow using the fastapi front end.
  start      Run a NeMo Agent Toolkit workflow using a front end configuration.
  uninstall  Uninstall a NeMo Agent Toolkit plugin packages from the local...
  validate   Validate a configuration file
  workflow   Interact with templated workflows.
```

Configuration file reference

Configuration file references

The configuration defines how the workflow operates, including functions, LLMs, and embedders along with general configuration settings. More details about the NeMo Agent toolkit workflow configuration file can be found [here](#).

- 1. General configuration (`general`):** The `general` section contains general configuration settings for NeMo Agent toolkit which is not specific to any workflow.
 - `use_uvloop` : Specifies whether to use `uvloop` event loop which can provide significant speedup. For debugging purposes it is recommended to set this to `false`.
 - `telemetry.logging` : Sets log level for logging.
 - `telemetry.tracing` : This is used in `config-tracing.yml` where `endpoint` is set to a Phoenix server. Traces of workflow can then be viewed in Phoenix UI.
- 2. Functions (`functions`):** The `functions` section contains the tools used in the workflow.
 - Preprocessing functions:
 - `cve_generate_vdbs` : Generates vector database from code repositories and documentation.
 - `agent_name` : Name of the agent executor (`cve_agent_executor`). Used to determine which tools are enabled in the agent to conditionally generate vector databases or indexes.
 - `embedder_name` : Name of embedder (`nim-embedder`) configured in `embedders` section.
 - `base_vdb_dir` : The directory used for storing vector database files. Default is `.cache/am_cache/vdb`.
 - `base_git_dir` : The directory for storing pulled git repositories used for code analysis. Default is `.cache/am_cache/git`.
 - `base_code_index_dir` : The directory used for storing code index files. Default is `./cache/am_cache/code_index`.
 - `cve_fetch_intel` : Fetches details about CVEs from NIST and CVE Details websites.
 - `max_retries` : Maximum number of retries on client and server errors. Default is 5.
 - `retry_on_client_errors` : Whether to retry on client errors. Default is true.
 - `request_timeout` : Timeout for individual HTTP requests in seconds. Default is 30.
 - `intel_source_timeout` : Timeout for each intel source (across all retries) in seconds. `null` means no timeout. Default is `null`.
 - `cve_process_sbom` : Prepares and validates input SBOM.
 - `cve_check_vuln_deps` : Cross-references every entry in the SBOM for known vulnerabilities. If none are found, the agent is skipped for that CVE.
 - `skip` : If true, skips this check and runs all CVEs through the agent. Useful when the input CVEs to the workflow have been validated by a dedicated vulnerability scanner. Default is false.
 - Core LLM engine functions:
 - `cve_checklist` : Generates tailored, context-sensitive task checklist for impact analysis.
 - `llm_name` : Name of LLM (`checklist_llm`) configured in `llms` section.
 - `llm_max_rate` : Controls the maximum LLM rate limit (requests per second). If not set or set to `null`, inherits from workflow-level `llm_max_rate`. If set to a number, uses that specific limit. Note: Setting to `null` only results in no rate limiting if the workflow-level setting is also `null`.
 - `Container Image Code QA System` : Retriever tool used by `cve_agent_executor` to query source code vector database.
 - `Container Image Developer Guide QA System` : Retriever tool used by `cve_agent_executor` to query documentation vector database.
 - `Lexical Search Container Image Code QA System` : Lexical search tool used by `cve_agent_executor` to search source code. This tool is an alternative to `Container Image Code QA System` and can be useful for very large code bases that take a long time to embed as a vector database. Disabled by default, enable by uncommenting the tool in `cve_agent_executor`.
 - `Internet Search` : SerpApi Google search tool used by `cve_agent_executor`.
 - `cve_agent_executor` : Iterates through checklist items using provided tools and gathered intel.
 - `llm_name` : Name of LLM (`cve_agent_executor_llm`) configured in `llms` section.
 - `tool_names` : Container Image Code QA System, Container Image Developer Guide QA System, (Optional) Lexical Search Container Image Code QA System, Internet Search
 - `llm_max_rate` : Controls the maximum LLM rate limit (requests per second) for all LLM calls made by the agent or its tools. If not set or set to `null`, inherits from workflow-level `llm_max_rate`. If set to a number, uses that specific limit. Note: Setting to `null` only results in no rate limiting if the workflow-level setting is also `null`.
 - `max_iterations` : The maximum number of iterations for the agent. Default is 10.
 - `prompt_examples` : Whether to include examples in agent prompt. Default is `false`.
 - `replace_exceptions` : Whether to replace exception message with custom message. Default is `true`.
 - `replace_exceptions_value` : If `replace_exceptions` is `true`, use this message. Default is `I do not have a definitive answer for this checklist item."`
 - `return_intermediate_steps` : Controls whether to return intermediate steps taken by the agent, and include them in the output file. Helpful for troubleshooting agent responses. Default is `false`.
 - `verbose` : Set to true for verbose output. Default is `false`.
 - `cve_summarize` : Generates concise, human-readable summarization paragraph from agent results.
 - `llm_name` : Name of LLM (`summarize_llm`) configured in `llms` section.
 - `llm_max_rate` : Controls the maximum LLM rate limit (requests per second). If not set or set to `null`, inherits from workflow-level `llm_max_rate`. If set to a number, uses that specific limit. Note: Setting to `null` only results in no rate limiting if the workflow-level setting is also `null`.
 - `cve_justify` : Assigns justification label and reason to each CVE based on summary.
 - `llm_name` : Name of LLM (`justify_llm`) configured in `llms` section.
 - `llm_max_rate` : Controls the maximum LLM rate limit (requests per second). If not set or set to `null`, inherits from workflow-level `llm_max_rate`. If set to a number, uses that specific limit. Note: Setting to `null` only results in no rate limiting if the workflow-level setting is also `null`.

- Postprocessing/Output functions
 - `cve_file_output` : Outputs workflow results to a file.
 - `file_path` : Defines the path to the file where the output will be saved.
 - `markdown_dir` : Defines the path to the directory where the output will be saved in individual navigable markdown files per CVE-ID.
 - `overwrite` : Indicates whether the output file should be overwritten when the workflow starts if it already exists. Will throw an error if set to `False` and the file already exists. Note that the overwrite behavior only occurs on workflow initialization. For pipelines started in HTTP mode, each new request will append the existing file until the workflow is restarted.
3. **LLMs (`llms`)**: The `llms` section contains the LLMs used by the workflow. Functions can reference LLMs in this section to use. The supported LLM API types in NeMo Agent toolkit are `nim` and `openai`. The models in this workflow use `nim`.
- Configured models in this workflow: `checklist_llm`, `code_vdb_retriever_llm`, `doc_vdb_retriever_llm`, `cve_agent_executor_llm`, `summarize_llm`, `justify_llm`
 - Each `nim` model is configured with the following attributes defined in the NeMo Agent toolkit's [NimModelConfig](#). Use [OpenAIModelConfig](#) for `openai` models.
 - `base_url` : Optional attribute to override `https://integrate.api.nvidia.com/v1`
 - `model_name` : The name of the LLM model used by the node.
 - `temperature` : Controls randomness in the output. A lower temperature produces more deterministic results.
 - `max_tokens` : Defines the maximum number of tokens that can be generated in one output step.
 - `top_p` : Limits the diversity of token sampling based on cumulative probability.
 - In addition, `nim` models in the Nemo Agent Toolkit also support the following built-in automatic retry configuration options defined by the [RetryMixin](#) class.
 - `do_auto_retry` : Whether to enable automatic retries.
 - `num_retries` : Number of times to retry a method call that fails with a retryable error.
 - `retry_on_status_codes` : List of HTTP status codes that should trigger a retry.
 - `retry_on_errors` : List of error messages that should trigger a retry. We recommend including both the error message and error code to accommodate libraries that don't expose a HTTP status code attribute in raised exceptions.
4. **Embedding models (`embedders`)**: The `embedders` section contains the embedding models used by the workflow. Functions can reference embedding models in this section to use. The supported embedding model API types in NeMo Agent toolkit are `nim` and `openai`.
- The models uses `nim` model, `nvidia/nv-embedqa-e5-v5`.
 - Each `nim` embedding model is configured with the following attributes defined in the NeMo Agent Toolkit's [NimEmbedderModelConfig](#). Use [OpenAIEmbedderModelConfig](#) for `openai` embedding models.
 - `base_url` : Optional attribute to override `https://integrate.api.nvidia.com/v1`
 - `model_name` : The name of the LLM model used by the node.
 - `truncate` : Specifies how inputs longer than the maximum token length of the model are handled. Passing `START` discards the start of the input. `END` discards the end of the input. In both cases, input is discarded until the remaining input is exactly the maximum input token length for the model. If `NONE` is selected, when the input exceeds the maximum input token length an error will be returned.
 - `max_batch_size` : Specifies the batch size to use when generating embeddings. We recommend setting this to 128 (default) or lower when using the cloud-hosted embedding NIM. When using a local NIM, this value can be tuned based on throughput/memory performance on your hardware.
 - In addition, `nim` models in the Nemo Agent Toolkit also support the following built-in automatic retry configuration options defined by the [RetryMixin](#) class.
 - `do_auto_retry` : Whether to enable automatic retries.
 - `num_retries` : Number of times to retry a method call that fails with a retryable error.
 - `retry_on_status_codes` : List of HTTP status codes that should trigger a retry.
 - `retry_on_errors` : List of error messages that should trigger a retry. We recommend including both the error message and error code to accommodate libraries that don't expose a HTTP status code attribute in raised exceptions.
5. **Workflow (`workflow`)**: The `workflow` section ties the previous sections together by defining the tools and LLM models to use in the workflow.
- `_type` : This is set to `cve_agent` indicating to NeMo Agent toolkit to use the function defined in [register.py](#) for the workflow.
 - `missing_source_action` : Action when source analysis is unavailable in the agent due to: missing source_info, VDB generation failures, or inaccessible repositories. Options are:
 - `error` : Fail pipeline with validation error
 - `skip_agent` : Collect intel and check dependencies, but skip agent
 - `continue_with_warning` : Run full pipeline with warning log (degraded analysis quality). Default is "continue_with_warning" for backwards compatibility.
 - `llm_max_rate` : Controls the maximum LLM rate limit (requests per second) globally for all workflow functions. Individual function `llm_max_rate` settings override this value. `null` means no rate limiting. Default is 5. This setting is particularly useful for:
 - Setting a baseline rate limit across all functions in the workflow
 - Preventing rate limit errors when using cloud-hosted models

Note: Setting `llm_max_rate: null` or omitting it at the function-level will inherit the workflow-level setting instead of disabling rate limiting. To disable rate limiting across all functions, set `llm_max_rate: null` or omit it at the workflow-level. As a proxy for disabling rate limiting for specific functions, set the function-level `llm_max_rate` to a high value. (Fully disabling rate limiting for individual functions is currently not supported.)

Example configuration:

```
functions:
  cve_checklist:
    # llm_max_rate not set - inherits global limit of 5 requests/second
  cve_agent_executor:
    llm_max_rate: null # Also inherits global limit of 5 requests/second
  cve_justify:
    llm_max_rate: 10 # Overrides global limit with 10 requests/second

workflow:
  _type: cve_agent
  llm_max_rate: 5 # Global limit of 5 requests/second for all functions
```

- The remaining configuration items correspond to attributes in [CVEWorkflowConfig](#) to specify the registered tools to use in the workflow.

6. **Evaluations and Profiling (eval)**: The `eval` section contains the evaluation settings for the workflow. Refer to [Evaluating NVIDIA Agent Intelligence Toolkit Workflows](#) for more information about NeMo Agent toolkit built-in evaluators as well as the plugin system to add custom evaluators. An example evaluation pipeline has been provided in the [Evaluation](#) section of this README. Additionally, the CVE workflow uses the `eval` section to configure a profiler that uses the NeMo Agent Toolkit evaluation system to collect usage statistics and store them to the local file system. You can find more information about NeMo Agent toolkit profiling and performance monitoring [here](#).

- `general.output_dir` : Defines the path to the directory where profiling results will be saved.
- `general.dataset` : Defines file path and format of dataset used to run profiling.
- `evaluators` : Custom evaluators defined in `eval/evaluators` . See [Evaluation](#) for more details on each custom evaluator and their configurations.
- `profiler` : The profiler for this workflow is configured with the following options.
 - `token_uniqueness_forecast` : Compute inter query token uniqueness
 - `workflow_runtime_forecast` : Compute expected workflow runtime
 - `compute_llm_metrics` : Compute inference optimization metrics
 - `csv_exclude_io_text` : Avoid dumping large text into the output CSV (helpful to not break structure)
 - `prompt_caching_prefixes` : Identify common prompt prefixes
 - `bottleneck_analysis` : Enable bottleneck analysis
 - `concurrency_spike_analysis` : Enable concurrency spike analysis. Set the `spike_threshold` to 7, meaning that any concurrency spike above 7 will be raised to the user specifically.

NGINX caching server

The docker compose file includes an `nginx-cache` proxy server container that enables caching for API requests made by the workflow. It is highly recommend to route API requests through the proxy server to reduce API calls for duplicate requests and improve workflow speed. This is especially useful when running the workflow multiple times with the same configuration (for example, for debugging) and can help keep costs down when using paid APIs.

The NGINX proxy server is started by default when running the `vuln-analysis` service. However, it can be started separately using the following command:

```
cd ${REPO_ROOT}
docker compose up --detach nginx-cache
```

To use the proxy server for API calls in the workflow, you can set environment variables for each base URL used by the workflow to point to `http://localhost:${NGINX_HOST_HTTP_PORT}/`. These are set automatically when running the `vuln-analysis` service, but can be set manually in the `.env` file as follows:

```
CWE_DETAILS_BASE_URL="http://localhost:8080/cwe-details"
DEPSDEV_BASE_URL="http://localhost:8080/depsdev"
FIRST_BASE_URL="http://localhost:8080/first"
GHSA_BASE_URL="http://localhost:8080/ghsa"
NGC_API_BASE="http://localhost:8080/nemo/v1"
NIM_EMBED_BASE_URL="http://localhost:8080/nim_embed/v1"
NVD_BASE_URL="http://localhost:8080/nvd"
NVIDIA_API_BASE="http://localhost:8080/nim_llm/v1"
OPENAI_API_BASE="http://localhost:8080/openai/v1"
OPENAI_BASE_URL="http://localhost:8080/openai/v1"
RHSA_BASE_URL="http://localhost:8080/rhsa"
SERPAPI_BASE_URL="http://localhost:8080/serpapi"
UBUNTU_BASE_URL="http://localhost:8080/ubuntu"
```

Customizing the Workflow

The primary method for customizing the workflow is to generate a new configuration file with new options. The configuration file defines the workflow settings, such as the functions, the LLM models used, and the output format. The configuration file is a YAML file that can be modified to suit your needs.

Customizing the SBOM input

To use an SBOM from a URL ([example](#)), update the SBOM info configuration to:

```
"sbom_info": {
  "_type": "http",
  "_url": "http://nvd.nist.gov/feeds/xml/cvss/v3/cvss-v3-metrics.xml"
}
```

```
        "url": "https://raw.githubusercontent.com/NVIDIA-AI-Blueprints/vulnerability-analysis/refs/heads/main/s:
    }
}
```

Customizing the LLM models

The workflow configuration file also allows customizing the LLM model and parameters for each component of the workflow, as well as which LLM API is used when invoking the model.

In any configuration file, locate the `llms` section to see the current settings. For example, the following snippet defines the LLM used for the default LLM model:

```
default_llm:
  _type: nim
  base_url: ${NVIDIA_API_BASE:-https://integrate.api.nvidia.com/v1}
  model_name: ${DEFAULT_MODEL_NAME:-meta/llama-3.1-70b-instruct}
  temperature: 0.0
  max_tokens: 2000
  top_p: 0.01
```

- `_type` : specifies the LLM API type. Refer to the [Supported LLM APIs](#) table for available options.
- `base_url` : Base URL for LLM. Sets to value of `NVIDIA_API_BASE` environment variable if set. Otherwise, sets to default NIM base URL. `base_url` can also be omitted in which case, the default base URL is used for type `nim` . We use an environment variable here so that we can easily set it to proxy server URL when running with `docker compose` .
- `model_name` : specifies the model name within the LLM API. Sets to value of `DEFAULT_MODEL_NAME` environment variable if set. Otherwise, sets to default model, `meta/llama-3.1-70b-instruct` . This is also applicable to the other LLM models in the workflow, each having their own environment variable for setting the model name. Refer to the LLM API documentation to determine the available models.
- `temperature` , `max_tokens` , `top_p` , ...: specifies the model parameters. The available parameters can be found in the NeMo Agent Toolkit [NIMModelConfig](#). Any non-supported parameters provided in the configuration will be ignored.

Instead of using a single default LLM model for the entire workflow, you can optionally use different LLM models and configurations for each function (e.g., checklist, agent, summarize, etc.).

To configure a custom LLM for a specific function:

- In the config file's `llms` section, uncomment the relevant LLM configuration (e.g., `checklist_llm` , `cve_agent_executor_llm` , `summarize_llm` , etc.)
- Update the model name and other settings as needed
- In the `functions` section, update the function's `llm_name` parameter to reference your custom LLM configuration

For example, to use a different model for the checklist function, uncomment the `checklist_llm` configuration.

```
llms:
  checklist_llm:
    _type: nim
    model_name: meta/llama-3.1-8b-instruct  # Different model
    temperature: 0.0

    max_tokens: 2000
    # ... other parameters
```

In the functions section, update the `llm_name` to reference the configuration we just updated.

```
functions:
  cve_checklist:
    _type: cve_checklist
    llm_name: checklist_llm  # Reference the LLM we just configured
```

Supported LLM APIs

Name	<code>_type</code>	Auth Env Var(s)	Base URL Env Var(s)	Proxy Server Route
NVIDIA Inference Microservices (NIMs) (Default)	<code>nim</code>	<code>NVIDIA_API_KEY</code>	<code>NVIDIA_API_BASE</code>	<code>/nim_llm/v1</code>
OpenAI	<code>openai</code>	<code>OPENAI_API_KEY</code>	<code>OPENAI_API_BASE</code> (used by langchain) <code>OPENAI_BASE_URL</code> (used by openai)	<code>/openai/v1</code>

Steps to configure an LLM model

- Obtain an API key and any other required auth info for the selected service.
- Update the `.env` file with the auth and base URL environment variables for the service as indicated in the [Supported LLM APIs](#) table. If you choose not to use the default LLM models in your workflow (`meta/llama-3.1-70b-instruct`), you can also add environment variables to override the model names to your `.env` file. In addition to `DEFAULT_MODEL_NAME` , you can also set `model_name` for the other LLM models using `CHECKLIST_MODEL_NAME` , `CODE_VDB_RETRIEVER_MODEL_NAME` , `DOC_VDB_RETRIEVER_MODEL_NAME` , `CVE_AGENT_EXECUTOR_MODEL_NAME` , `SUMMARIZE_MODEL_NAME` , and `JUSTIFY_MODEL_NAME` .
- Update the config file as described above. For example, if you want to use OpenAI's `gpt-4o` model for checklist generation, update

checklist_llm in the llms section to:

```
checklist_llm:
  _type: openai
  model_name: ${CHECKLIST_MODEL_NAME:-gpt-4o}
  temperature: 0.0
  seed: 0
  top_p: 0.01
  max_retries: 5
```

Please note that the prompts have been tuned to work best with the Llama 3.1 70B NIM and that when using other LLM models it may be necessary to adjust the prompting.

Customizing the embedding model

Vector databases are used by the agent to fetch relevant information for impact analysis investigations. The embedding model used to vectorize your documents can significantly affect the agent's performance. The default embedding model used by the workflow is the NIM [nvidia/nv-embedqa-e5-v5](#) model, but you can experiment with different embedding models of your choice.

To test a custom embedding model, modify the workflow configuration file (for example, `config.yml`) in the `embedders` section. For example, the following snippet defines the settings for the default embedding model. The full set of available parameters for a NIM embedder can be found [here](#).

```
nim_embedder:
  _type: nim
  base_url: ${NVIDIA_API_BASE:-https://integrate.api.nvidia.com/v1}
  model_name: ${EMBEDDER_MODEL_NAME:-nvidia/nv-embedqa-e5-v5}
  truncate: END
  max_batch_size: 128
```

- `_type` : specifies the LLM API type. Refer to the [Supported LLM APIs](#) table for available options.
- `base_url` : Base URL for embedding model. Sets to value of `NVIDIA_API_BASE` environment variable if set. Otherwise, sets to default NIM base URL. `base_url` can also be omitted in which case, the default base
- `model_name` : specifies the model name for the embedding provider. Sets to value of `EMBEDDER_MODEL_NAME` environment variable if set. Otherwise, sets to default embedding model, `nvidia/nv-embedqa-e5-v5` . Refer to the embedding provider's documentation to determine the available models.
- `truncate` : specifies how inputs longer than the maximum token length of the model are handled. Passing `START` discards the start of the input. `END` discards the end of the input. In both cases, input is discarded until the remaining input is exactly the maximum input token length for the model. If `NONE` is selected, when the input exceeds the maximum input token length an error will be returned.
- `max_batch_size` : specifies the batch size to use when generating embeddings. We recommend setting this to 128 (default) or lower when using the cloud-hosted embedding NIM. When using a local NIM, this value can be tuned based on throughput/memory performance on your hardware.

Steps to configure an alternate embedding provider

- If using OpenAI embeddings, first obtain an API key, then update the `.env` file with the auth and base URL environment variables for the service as indicated in the [Supported LLM APIs](#) table. Otherwise, proceed to step 2. If you choose not to use the default embedding model (`nvidia/nv-embedqa-e5-v5`), you can also add `EMBEDDER_MODEL_NAME` to your `.env` file to override the default.
- Update the `embedders` section of the config file as described above.

Example OpenAI embedding configuration:

```
nim_embedder:
  _type: openai
  model_name: ${EMBEDDER_MODEL_NAME:-text-embedding-3-small}
  max_retries: 5
```

For OpenAI models, only a subset of parameters are supported. The full set of available parameters can be found in the config definitions [here](#). Any non-supported parameters provided in the configuration will be ignored.

The current workflow uses [FAISS](#) to create the vector databases. Interested users can customize the source code to use other vector databases such as [cuVS](#).

Customizing the Output

Currently, there are 3 types of outputs supported by the workflow:

- File output:** The output data is written to a file in JSON format.
- HTTP output:** The output data is posted to an HTTP endpoint.
- Print output:** The output data is printed to the console.

To customize the output, modify the workflow configuration file accordingly. Locate the `workflow` section to see the output destination used by the workflow. For example, in the configuration file `configs/config.yml` , the following snippet from the `functions` section defines the function that writes the workflow output as a single json file and individual markdown files per CVE-ID:

```
cve_file_output:
  _type: cve_file_output
```



```
file_path: .tmp/output.json
markdown_dir: .tmp/vulnerability_markdown_reports
overwrite: True
```

The following snippet from the `workflow` section then configures the workflow to use the above function for output:

```
workflow:
  _type: cve_agent
  ...
  cve_output_config_name: cve_file_output
```

To post the output to an HTTP endpoint, you can add the following to the `functions` section of the config file, replacing the domain, port, and endpoint with the desired destination (note the trailing slash in the "url" field). The output will be sent as JSON data.

```
cve_http_output:
  _type: cve_http_output
  url: http://<domain>:<port>/
  endpoint: "<endpoint>"
```

The workflow can then be updated to use the new function:

```
workflow:
  _type: cve_agent
  ...
  cve_output_config_name: cve_http_output
```

Additional output options will be added in the future.

Adding Test Time Compute (TTC)

Test time compute (TTC) is a technique that can improve agentic workflow performance by allocating more compute at inference time. There are a variety of available TTC strategies. This blueprint includes an implementation of the **majority voting** TTC strategy using the NeMo Agent Toolkit's [Test Time Compute](#) module. Majority voting is a simple strategy that executes the workflow multiple times, then takes a majority vote on the output, using the most frequent justification status as the final output status. This improves both the accuracy and the consistency of the workflow predictions.

To run the workflow with the default majority voting test time compute settings, **it's important to first disable LLM caching** to avoid voting on identical results each time (see [Disable Caching](#) below). Once LLM caching is disabled, use the `nat run` command as usual, while passing in the [config-ttc.yml](#) configuration file:

```
nat run --config_file=configs/config-ttc.yml --input_file=data/input_messages/morpheus:23.11-runtime.json
```

This will run the workflow multiple times (3 times by default in `config-ttc.yml`) and return the first execution output for each CVE that matches the majority vote.

Reviewing the TTC output

By default, the output artifacts produced by TTC are identical to those of the regular workflow (e.g. `output.json` , markdown reports, workflow return object). See the [Reviewing the workflow output](#) section for more info. The key difference when running TTC is that it improves the accuracy and consistency of the *predictions* in the output (i.e., affected status) by taking the majority vote.

Customizing the TTC configuration

To customize test time compute, modify the [config-ttc.yml](#) configuration file accordingly.

First, modify the `ttc_strategies` section of the config file as needed:

```
ttc_strategies:
  selection_strategy_majority_voting:
    _type: majority_voting_selection
    selection_mode: decisive
```

- `selection_strategy_majority_voting` : The name of the TTC strategy that can be referenced in the workflow configuration below.
- `_type` : Specifies the type of TTC strategy. Currently only `majority_voting_selection` is implemented.
- `selection_mode` : Determines how the majority vote is calculated. Two modes are available (Default is `decisive`):
 - `simple` : A standard majority vote where ties are marked as `UNKNOWN` .
 - `decisive` : Ignores majority `UNKNOWN` votes in favor of the next most frequent label, except in the case of a tie. (This helps penalize `UNKNOWN` labels, which can be overrepresented in model outputs relative to real-world distributions.)

Next, modify the `workflow` section of the configuration as needed:

```
workflow:
  _type: execute_then_select_function
  selector: selection_strategy_majority_voting
  augmented_fn: cve_agent_workflow
  output_fn: cve_file_output
  num_executions: 3
  max_concurrency: 1
  early_stop_threshold: false
```

- `_type` : Indicates the test time compute orchestration function to be run in the workflow. The `execute_then_select_function` is responsible for executing the workflow multiple times, then running the `selector` to choose the final result.
- `selector` : The name of the test time compute selector function used to select the final result. This must match a strategy name defined in the `ttc_strategies` section. Here, `selection_strategy_majority_voting` uses the majority voting selector.
- `augmented_fn` : The name of the workflow function to be executed multiple times. This must match a function defined in the `functions` section. Here, we reference the main `cve_agent_workflow` function.
- `output_fn` : The name of the output function to use in order to save the TTC output. This must match a function defined in the `functions` section. Set to `null` to not save any output. Default is `cve_file_output`.
- `num_executions` : The maximum number of times to execute the workflow before voting occurs. Must be at least 1. Default is 3.
- `max_concurrency` : The number of workflows to run simultaneously. Default is 1.
- `early_stop_threshold` : If set to an integer, n , stops executing the workflow for a given CVE after n consecutive identical justification statuses are observed, improving efficiency for stable outputs. If `false`, always runs the full `num_executions` for all CVEs. The early stopping threshold number must be strictly less than the `num_executions`. Default is `false`.

 Note

Since TTC executes the workflow multiple times, it's recommended to disable the output in the base workflow by setting `cve_agent_workflow.cve_output_config_name` to null to avoid generating output for each execution. Instead, configure the output function by setting `output_fn` in the `execute_then_select_function` to generate output for the final results from TTC.

 Warning

Increasing `num_executions` typically boosts accuracy and consistency. However, it also increases cost in LLM calls, token usage, and latency. It is important to benchmark your specific use case to find the optimal balance between accuracy, performance, and cost. Consider starting with `num_executions: 3` and incrementing based on your accuracy requirements. Also consider enabling early stopping to reduce costs when consecutive runs produce stable results.

Note: When using NVIDIA-hosted model endpoints, you may encounter `429: Too Many Requests` errors with `max_concurrency` greater than 1. See [429 Errors - Too Many Requests](#) for solutions.

Writing a custom TTC strategy

You can also implement custom TTC strategies other than majority voting by following the NeMo Agent Toolkit [Test Time Compute reference](#).

Evaluating the Workflow

Similar to unit tests in software development, evaluation is an essential part of developing agentic workflows. As the agentic workflow is modified during development, evaluation pipelines enable a structured way of ensuring performance. In this section, we use NeMo Agent toolkit's [evaluation](#) feature to implement an evaluation pipeline for the vulnerability analysis workflow. The core behavior of the vulnerability analysis workflow can be assessed by evaluating the two key metrics of `accuracy` and `consistency`. Advanced developers can add their own [custom evaluators](#) that target other metrics.

 Note

The evaluation pipeline is not a part of the stable core workflow API, and may be updated in future releases as agentic workflow evaluation best practices evolve.

Running Evaluation (Overview)

To run the evaluation pipeline, use the `nat eval` command with the `config-eval.yml` configuration file.

```
nat eval --config_file=configs/config-eval.yml
```



This will run evaluation for accuracy on the workflow's output fields of `status` and `label`. Results can be viewed in `.tmp/evaluators`, in the files with the suffix "accuracy_output."

Accuracy is calculated by comparing the current workflow run's results against a ground truth answer key, known as an evaluation dataset. A mock evaluation dataset is given. Developers looking to use the evaluation pipeline should provide their own evaluation datasets; see [Inputting Evaluation Datasets](#) for details. More details on the implementation and outputs of the accuracy evaluator can be found in the [Accuracy Evaluator](#) section.

To measure the workflow's consistency, we examine the results of the workflow across multiple runs. To enable this evaluator, uncomment the last section of `config-eval.yml`. Then, follow the [Disable Caching](#) section of the README to disable caching, and run the workflow with the `--reps` flag to enable multiple repetitions. Results can be viewed in `.tmp/evaluators`, in the files with the suffix "consistency_output."

```
nat eval --config_file=configs/config-eval.yml --reps=3
```



More details on running with repetitions can be found in the [Running Multiple Evaluation Repetitions](#) section. More details on the implementation and outputs of the consistency evaluator can be found in the [Consistency Evaluator](#) section.

For advanced developers looking to write custom evaluators to measure other metrics, see the [Writing Custom Evaluators](#) section.

Inputting Evaluation Datasets

Evaluation datasets containing ground truth labels should be put in the `data/eval_datasets/` directory. A test set should be a JSON file with the following structure:

```
{
  "dataset_id": "...",
  "dataset_description": "...",
  "containers": {
    "containerA": {
      "container_image": {
        "name": "...",
        "tag": "...",
        "source_info": [...],
        "sbom_info": {...}
      },
      "ground_truth": [
        {
          "vuln_id": "CVE-XXXX-XXXX",
          "status": "NOT AFFECTED",
          "label": "..."
        }
      ]
    },
    "containerB": {
      "container_image": {...},
      "ground_truth": [...]
    }
  }
}
```

Multiple JSON files can be created to represent different test sets, which is useful for organization when running different evaluation experiments. The test set to be used is configurable via the `eval.general.dataset.filepath` field.

A mock test set `eval_dataset.json` is provided, which includes two containers (`morpheus:23.11-runtime` and `morpheus:24.03-runtime`) that have a few CVEs each. Each entry in the `containers` field consists of container metadata, along with a list of CVEs and their corresponding ground truth labels.

A parsing script (`eval/parse_eval_input.py`) is used to transform test sets into a format ingestible by the workflow. Advanced developers can choose to modify the input test set structure by creating their own parsing script. The path to this script is configurable at `eval.general.dataset.function` . Reference the NeMo Agent Toolkit documentation on [Custom Dataset Format](#) to learn more.

Additionally, the output directory of evaluation results is configurable via the `eval.general.output_dir` field.

 Note

For large input test sets, developers may encounter `429: Too Many Requests` errors if using NVIDIA-hosted model endpoints. See [Troubleshooting](#) for how to mitigate this.

Running Multiple Evaluation Repetitions

The NeMo Agent Toolkit provides a `--reps` flag, which can be used to conveniently execute an evaluation run multiple times. This is useful for capturing metrics such as workflow consistency. Note that **caching must be disabled first** (see [Disable Caching](#) below) in order for

evaluation repetitions to re-run the workflow instead of relying on cached results.

Below is an example of how to run an evaluation experiment with three repetitions:

```
nat eval --config_file=configs/config-eval.yml --reps=3
```

 Note

Running multiple repetitions will take extra time. Additionally, developers may encounter `429: Too Many Requests` errors if using hosted model endpoints, due to the concurrency introduced when running multiple repetitions. See [Troubleshooting](#) for how to mitigate this.

Disable Caching

LLM caching should be disabled for evaluation with multiple reps to measure the workflow's accuracy distribution and consistency. Two options for disabling caching are provided below.

Option 1: Disable Caching Globally

Globally disable all LLM caching by setting the `NVIDIA_API_BASE` variable to directly access the hosted model endpoint instead of going through NGINX.

```
export NVIDIA_API_BASE=https://integrate.api.nvidia.com/v1
```

Re-enable caching by setting the `NVIDIA_API_BASE` variable back to NGINX.

```
export NVIDIA_API_BASE=http://nginx-cache/nim_llm/v1
```

Option 2: Disable Caching on Specific Components

This option enables developers to disable caching for specific parts of the workflow. This approach is useful for in-depth evaluation experiments; for example, a developer may want to disable caching for just the `justify_llm` in order to test consistency of the justification

part of the workflow without re-running everything before that point.

In `config-eval.yml` , assign the `base_url` field of the target components to `"https://integrate.api.nvidia.com/v1"` . We recommend creating multiple config files to keep track of different evaluation experiments. To re-enable caching, simply assign the `base_url` back to `${NVIDIA_API_BASE:-https://integrate.api.nvidia.com/v1}` .

For quick testing, developers can also change configuration in the CLI using the `--override` flag, which uses dot notation. This change will not persist across future runs. Below is an example that disables caching on the `checklist_llm` .

```
nat eval --config_file=configs/config-eval.yml --reps=3 --override llms.checklist_llm.base_url https://integrate.:
```

Accuracy and Consistency Evaluators

In this section, the implementation of the two evaluators for accuracy and consistency are discussed in detail.

Accuracy Evaluator

For the vulnerability analysis agentic workflow, the most basic method to assess workflow performance is to measure accuracy of the `status` and `label` fields.

Each CVE has a ground truth `status` field, as well as a `label` field that corresponds to the status. Valid statuses include: `AFFECTED` or `NOT AFFECTED` . Valid labels include: `vulnerable` if the CVE status is `AFFECTED` , or one of 10 VEX statuses if the status is `NOT AFFECTED` (see **Justification Assignment** under the [Key Components section](#)).

For each CVE, if the workflow's output matches the ground truth label, a score of 1 is assigned. Otherwise, a score of 0 is assigned. The evaluator then takes the mean of all CVEs' scores to produce an accuracy score between 0 and 1.

Configuration Options

The accuracy evaluator is implemented in `eval/evaluators/accuracy.py` and registered in NeMo Agent Toolkit with the unique identifier (`_type`) of `accuracy` .

The evaluator takes in a target field (`status` or `label`), which can be configured in `config-eval.yml` under `evaluators.[evaluator name].field` . This design follows a common NeMo Agent Toolkit design pattern to promote code reusability. In `config-eval.yml` , a `status_accuracy_evaluator` and a `label_accuracy_evaluator` are set up as two separate evaluators, but under the hood, they share the same `_type` and therefore the same source code.

The evaluator also takes in a `duplicates_policy` field. This is to account for instances where, if an input set is an aggregation of ground truths from different sources, the set may contain duplicate CVEs with conflicting answers. For example, a CVE may be listed twice in the dataset as both `AFFECTED` and `NOT AFFECTED` , with different label justifications. We provide three policies for handling duplicates:

- `drop_all` : Any CVE with conflicting ground truth will be omitted and not contribute towards evaluation results.
- `keep_first` : Use the first encountered instance of a CVE as the truth, disregard all other instances.
- `keep_all` : Any answer listed in the ground truth for a CVE is considered valid (eg. if the ground truth has two listings of a CVE, with one being `AFFECTED` and the other being `NOT AFFECTED` , either answer given by the pipeline will be considered correct).

Output Format

The final scores are written to the files `status_accuracy_output.json` and `label_accuracy_output.json` . The output will include an `average_score` overview field, and a an `eval_output_items` field which gives a more detailed breakdown.

average_score Field

The `average_score` field contains a dict with metrics corresponding to `pandas.DataFrame.describe` output. The goal of this field is to aggregate the accuracy score across all runs, e.g., when using the `--reps` option. If only one run is performed, `count` and `mean` would be the only meaningful metrics; for example, if the run yielded accuracy of 0.8, the `average_score` field would look like this:

```
"average_score": {
  "count": 1,
  "mean": 0.8,
  "std": null,
  "min": 0.8,
  "25%": 0.8,
  "50%": 0.8,
  "75%": 0.8,
  "max": 0.8
}
```

Meanwhile, if there were multiple runs, the `average_score` field would capture the accuracy distribution across runs. For example, with 5 runs, the output might look like this:

```
"average_score": {
  "count": 5,
  "mean": 0.74,
  "std": 0.07,
  "min": 0.65,
  "25%": 0.7,
  "50%": 0.75,
  "75%": 0.8,
  "max": 0.8
}
```


A script is provided in `eval/visualizations/box_and_whisker_plot.py` to visualize this distribution as a box and whisker plot. It is especially helpful for comparing the plots of multiple experiments side-by-side, or comparing `status` accuracy against `label` accuracy. It can be run like so:

```
python3 src/vuln_analysis/eval/visualizations/box_and_whisker_plot.py \
.tmp/evaluators/accuracy-output-1.json \
.tmp/evaluators/accuracy-output-2.json \
--title "Accuracy: Experiment 1 vs. Experiment 2" \
--save src/vuln_analysis/eval/visualizations/my-plot.png \
--labels my-experiment1 my-experiment2
```

Users can list any number of output files, as well as configure chart title, save location, and X-axis labels.

eval_output_items Field

The `eval_output_items` field gives a more detailed output of the experiment. It lists the runs, giving each run a unique `id`, a `score` (the accuracy score for that run), and a `reasoning` field.

This last field breaks the run down by container, and lists the specific CVEs that were run in each container (`question`), the ground truth answers (`[target field]_answer`), and the answers generated by the workflow (`generated_[target field]_answer`).

Consistency Evaluator

This evaluator measures the consistency of results across repeated runs. It can be configured to measure consistency of the `status` or `label` fields.

Consistency is defined as: $\sum p^2$ per CVE, where p is the probability of each unique status or label. For example, if the status for a CVE across 3 runs are (AFFECTED, NOT_AFFECTED, NOT_AFFECTED), then the probabilities become $p(\text{AFFECTED}) = 1/3$, $p(\text{NOT_AFFECTED}) = 2/3$. Therefore, the consistency score is $(1/3)^2 + (2/3)^2 = 0.56$. A score of 1.0 means perfect consistency (all runs yielded the exact same values). The evaluator outputs a consistency score for each CVE.



Note

This evaluator should only be used if evaluation is being run with [multiple repetitions](#). An error will occur if this criteria is not met. To enable this evaluator, uncomment the last section in `config-eval.yml`.

Configuration Options

The accuracy evaluator is implemented in `eval/evaluators/consistency.py` and registered in NeMo Agent Toolkit with the unique identifier (`_type`) of `consistency`.

The evaluator takes in a configurable target field (`status` or `label`), following a similar design pattern as the [Simple Accuracy Evaluator](#)

Output Format

The final scores are written to the files `status_consistency_output.json` and `label_consistency_output.json`.

The output will include an `average_score` field which gives the average consistency score across all containers, and an `eval_output_items` field which lists the input containers.

For each container in the `eval_output_items` field, there is an unique `id` and a consistency `score`, which is the mean of the consistency scores of each CVE within that container. There is also a `reasoning` field that displays the following details:

- `status_consistencies`: consistency score for a specific CVE
- `total_cves`: the total number of CVEs in the container
- `num_reps`: number of evaluation repetitions that were run

Writing Custom Evaluators

A custom evaluator consists of three components, an evaluator configuration, the evaluator's core logic, and registration of the evaluator into the toolkit.

Configuration: The evaluator must have a [configuration](#) class that defines the evaluator `name` and any evaluator-specific parameters. The evaluator `name` is a unique identifier that the evaluator will be registered under in the NeMo Agent toolkit. During configuration of an evaluation pipeline (for example, in `config-eval.yml`), the `name` corresponds to the `_type` field. The configuration class should inherit from NeMo Agent toolkit's `EvaluatorBaseConfig` class.

Evaluator Core Logic: The main logic of the evaluator should be implemented in an async method. This method should take in an `EvalInput` (which contains a list of `EvalInputItem` objects), and output an `EvalOutput` (which contains a list of `EvalOutputItem` objects). Read more about these data structures [here](#).

Evaluator Registration: Finally, like all functions used in the NeMo Agent toolkit, the custom evaluator must be registered within the toolkit. An async registration function should be decorated with the `register_evaluator` decorator, which takes in the evaluator configuration class defined earlier. To ensure that the evaluator is registered at runtime, import the evaluator function in the `register.py` file. The following command will list all registered evaluators.

```
nat info components -t evaluator
```

Below is a skeleton example showing the structure of a custom evaluator:

```
from nat.builder.builder import EvalBuilder
from nat.builder.evaluator import EvaluatorInfo
from nat.cli.register_workflow import register_evaluator
from nat.data_models.evaluator import EvaluatorBaseConfig
from nat.eval.evaluator.evaluator_model import EvalInput, EvalOutput, EvalOutputItem
from pydantic import Field

# Step 1: Define the evaluator configuration
class MyCustomEvaluatorConfig(EvaluatorBaseConfig, name="my_custom_evaluator"):
    my_custom_parameter: float = Field(default=0.5, description="A custom parameter")

# Step 2: Register the evaluator with the register_evaluator decorator
@register_evaluator(config_type=MyCustomEvaluatorConfig)
async def my_custom_evaluator(config: MyCustomEvaluatorConfig, builder: EvalBuilder):
    # Instantiate the evaluator class with config parameters
    evaluator = MyCustomEvaluator(
        max_concurrency=builder.get_max_concurrency(),
        my_custom_parameter=config.my_custom_parameter
    )
    # Yield EvaluatorInfo with the evaluate function
    yield EvaluatorInfo(
        config=config,
        evaluate_fn=evaluator.evaluate,
        description="My Custom Evaluator"
    )

# Step 3: Implement the evaluator core logic
class MyCustomEvaluator:

    def __init__(self, max_concurrency: int, my_custom_parameter: float):
        self.max_concurrency = max_concurrency
        self.my_custom_parameter = my_custom_parameter

    async def evaluate(self, eval_input: EvalInput) -> EvalOutput:
        eval_output_items = []

        # Handle each item in EvalInput
        for item in eval_input.eval_input_items:
            # Your custom evaluation logic here
            eval_output_items.append(
                EvalOutputItem(id=item.id, score=score, reasoning=reasoning)
            )

        # Calculate aggregate score
        avg_score = 0.0 # Your aggregation logic here

        # return an EvalOutput
        return EvalOutput(average_score=avg_score, eval_output_items=eval_output_items)
```

Importing the custom evaluator in `register.py` to ensure it's registered at runtime can be done like so:

```
# In register.py

from vuln_analysis.eval.evaluators.my_custom_evaluator import my_custom_evaluator
```

For developers looking to build custom evaluators, we strongly recommend reviewing the full NeMo Agent toolkit [evaluation documentation](#), as well as the NeMo Agent toolkit documentation for writing [custom evaluators](#). Developers can also reference the source code of the `accuracy.py` and `consistency.py` evaluators inside the `eval/evaluators` directory.

Troubleshooting

Several common issues can arise when running the workflow. Here are some common issues and their solutions.

Git LFS issues

If you encounter issues with Git LFS, ensure that you have Git LFS installed and that it is enabled for the repository. You can check if Git LFS is enabled by running the following command:

```
git lfs install
```

Verifying that all files are being tracked by Git LFS can be done by running the following command:

```
git lfs ls-files
```

Files which are missing will show a `-` next to their name. To ensure all LFS files have been pulled correctly, you can run the following command:

```
git lfs fetch --all
git lfs checkout *
```

Container build issues

When building containers for self-hosted NIMs, certain issues may occur. Below are common troubleshooting steps to help resolve them.

Device error

If you encounter an error resembling the following during the [container build process for self-hosted NIMs](#):

```
nvidia-container-cli: device error: {n}: unknown device: unknown
```

This error typically indicates that the container is attempting to access GPUs that are either unavailable or non-existent on the host. To resolve this, verify the GPU count specified in the [docker-compose.nim.yml](#) configuration file:

- Navigate to the `deploy.resources.reservations.devices` section and check the count parameter.
- Set the environment variable `NIM_LLM_GPU_COUNT` to the actual number of GPUs available on the host machine before building the container. Note that the default value is set to 4.

This adjustment ensures the container accurately matches the available GPU resources, preventing access errors during deployment.

Deploy.Resources.Reservations.devices error

If you encounter an error resembling the following during the [container build process for self-hosted NIMs](#) process:

```
1 error(s) decoding:

* error decoding 'Deploy.Resources.Reservations.devices[0]': invalid string value for 'count' (the only value allowed is the number zero)
```

This is likely caused by an [outdated Docker Compose version](#). Please upgrade Docker Compose to at least `v2.21.0`.

NGINX caching server

Because the workflow makes such heavy use of the caching server to speed up API requests, it is important to ensure that the server is running correctly. If you encounter issues with the caching server, you can reset the cache.

Resetting the entire cache

To reset the entire cache, you can run the following command:

```
docker compose down -v
```

This will delete all the volumes associated with the containers, including the cache.

Resetting just the LLM cache or the services cache

If you want to reset just the LLM cache or the services cache, you can run the following commands:

```
docker compose down

# To remove the LLM cache
docker volume rm ${COMPOSE_PROJECT_NAME:-vuln_analysis}_llm-cache

# To remove the services cache
docker volume rm ${COMPOSE_PROJECT_NAME:-vuln_analysis}_service-cache
```

Vector databases

We've integrated VDB and embedding creation directly into the workflow with caching included for expediency. However, in a production environment, it's better to use a separately managed VDB service.

NVIDIA offers optimized models and tools like NIMs ([build.nvidia.com/explore/retrieval](#)) and cuVS ([github.com/rapidsai/cuvs](#)).

Service errors

National Vulnerability Database (NVD)

These typically resolve on their own. Please wait and try running the workflow again later. Example errors:

404

```
Error requesting [1/10]: (Retry 0.1 sec) https://services.nvd.nist.gov/rest/json/cves/2.0: 404, message='', url=URL
```

503

```
Error requesting [1/10]: (Retry 0.1 sec) https://services.nvd.nist.gov/rest/json/cves/2.0: 503, message='Service Unavailable'
```

NVIDIA API Catalog / NVIDIA-hosted NIMs

429 Errors - Too Many Requests

`Exception: [429] Too Many Requests` can occur when using NVIDIA-hosted model endpoints, if your requests exceed the rate limit.

Quick Fix:

To resolve 429 errors, try reducing the rate of LLM requests. This involves adjusting `llm_max_rate` (controls request rate within a workflow) and, for certain operations, `max_concurrency` (controls how many workflows run in parallel).

Start by reducing `llm_max_rate` from the default of 5, for example:

```
workflow:
  llm_max_rate: 3 # Reduce from default of 5 to 3 requests per second
```

Where to make this change:

- **Standard workflows** ([config.yml](#)): Reduce `workflow.llm_max_rate`
- **Test Time Compute** ([config-ttc.yml](#)): Reduce `cve_agent_workflow.llm_max_rate` , and set `workflow.max_concurrency: 1` to limit parallelization
- **Evaluation** ([config-eval.yml](#)): Reduce `workflow.llm_max_rate` , and set `eval.general.max_concurrency: 1` to limit parallelization

Understanding Rate Limiting:

For more control and optimization beyond the quick fix above, you can fine-tune rate limiting at multiple levels:

1. Control request rate within a single workflow execution:

- Set a global `llm_max_rate` in workflows or functions with `_type: cve_agent` (e.g., `workflow.llm_max_rate: 5` in [config.yml](#) or `cve_agent_workflow.llm_max_rate: 5` in [config-ttc.yml](#)) to limit the rate (requests per second) of LLM requests
- Set individual function `llm_max_rate` values (e.g., `cve_justify.llm_max_rate: 10`) to override the global setting for functions that make LLM calls: `cve_checklist` , `cve_agent_executor` , `cve_summarize` , `cve_justify` . This is useful when functions use model endpoints with varying rate limits.

See [Configuration file reference](#) for more details on `llm_max_rate` configuration.

2. Control parallelism across multiple workflow executions:

Certain operations parallelize multiple workflow executions simultaneously, which can quickly exceed rate limits:

- **Evaluation** (`nat eval`): Parallelizes evaluation input items and evaluation repetitions. Set `eval.general.max_concurrency` to a low value such as 1 to reduce parallelization.
- **Test Time Compute**: Parallelizes runs across multiple TTC repetitions. Set `workflow.max_concurrency` in workflows with `_type: execute_then_select_function` (e.g., in [config-ttc.yml](#)) to a low value such as 1 to reduce parallelization.

 Note

Reducing `llm_max_rate` and `max_concurrency` will result in lower throughput when processing large workloads (e.g., many CVEs, large evaluation test sets, or many repetitions). For production use cases requiring higher throughput, we recommend switching to [self-hosting](#).

Authentication errors

Authentication errors will occur if required API key(s) are invalid or have not been set as environment variables as described in [Set up the environment file](#). For example, the following error will occur if `NVIDIA_API_KEY` is not properly set:

```
Error: [401] Unauthorized
Authentication failed
```

Note that exporting the required environment variables in a container shell will not persist outside of that shell. Instead, we recommend shutting down the containers (`docker compose down`), [setting the required environment variables](#), and then [starting the containers](#) again.

Controlling NAT error log verbosity

When errors occur in NAT functions (such as LLM calls or agent execution), the error logs include the input data that was being processed. For large inputs, this can result in extremely long error messages. You can control the verbosity of these error logs using the `NAT_ERROR_LOG_MAX_LENGTH` environment variable.

Setting the environment variable:

Update your `.env` file with the desired value, for example:

```
# Truncate error logs to 1000 characters (default is 500)
NAT_ERROR_LOG_MAX_LENGTH=1000
```

To disable NAT log truncation, set the value to 0. This can be helpful for debugging.

```
# Disable truncation (show full error messages)
NAT_ERROR_LOG_MAX_LENGTH=0
```

Then, shut down the containers (`docker compose down`), [export the variables in your `.env` file](#), and [start the containers](#) again.

macOS Workarounds

Testing and development of the workflow may be possible on macOS, however macOS is not officially supported or tested for this blueprint. Platform differences may require extra troubleshooting or impact performance. (See [Operating system requirements](#) for more info.)

For users wishing to test on macOS, please see below for a list of known issues and workarounds. Additional user-reported issues and workarounds can be found by searching [open issues for macOS](#).

Embedding API Connection Errors

Error:

`ConnectionError` / `RemoteDisconnected` error that occurs during vector database (VDB) generation.

```
requests.exceptions.ConnectionError: ('Connection aborted.', RemoteDisconnected('Remote end closed connection with'))
```

Cause:

This error can occur during VDB generation when the embedding batch size causes the request to exceed the NVIDIA-hosted API's timeout limits. On macOS, the default `max_batch_size` of 128 can result in a payload too large to process in a single HTTP request within the timeout.

Solution:

Try reducing the embedding batch size in your configuration file:

```
embedders:
  nim_embedder:
    max_batch_size: 32 # Reduced from default of 128
```

This change should be made in all configuration files you plan to use (e.g., `config.yml`, `config-eval.yml`, `config-ttc.yml`).

Testing and validation

Test-driven development is essential for building reliable LLM-based agentic systems, especially when deploying or scaling them in production environments.

In our development process, we use the Morpheus public container as a case study. We perform security scans and collaborate with developers and security analysts to assess the exploitability of identified CVEs. Each CVE is labeled as either vulnerable or not vulnerable. For non-vulnerable CVEs, we provide a justification based on one of the ten VEX statuses. Team members document their investigative steps and findings to validate and compare results at different stages of the system.

We have collected labels for 38 CVEs, which serve several purposes:

- Human-generated checklists, findings, and summaries are used as ground truth during various stages of prompt engineering to refine LLM output.
- The justification status for each CVE is used as a label to measure end-to-end workflow accuracy. Every time there is a change to the system, such as adding a new agent tool, modifying a prompt, or introducing an engineering optimization, we run the labeled dataset through the updated workflow to detect performance regressions.

As a next step, we plan to integrate this process into our CI/CD pipeline to automate testing. While LLMs' non-deterministic nature makes it difficult to assert exact results for each test case, we can adopt a statistical approach, where we run the workflow multiple times and ensure that the average accuracy stays within an acceptable range.

We recommend that teams looking to test or optimize their CVE analysis system curate a similar dataset for testing and validation. Note that in test-driven development, it's important that the model has not achieved perfect accuracy on the test set, as this may indicate overfitting or that the set lacks sufficient complexity to expose areas for improvement. The test set should be representative of the problem space, covering both scenarios where the model performs well and where further refinement is needed. Investing in a robust dataset ensures long-term reliability and drives continued performance improvements.

Roadmap

- ☒ Configurable option for skipping the `VulnerableDependencyChecker`
- ☒ Configurable retries and timeouts for intel fetching
- ☒ Upgrade to [NAT v1.2](#)
- ☒ Configurable NIM error handling
- ☒ Configurable missing source info and VDB handling
- ☒ [NAT Evaluation](#) integration
- ☐ [NAT W&B Weave](#) integration
- ☒ [NAT Test Time Compute](#) integration
- ☒ Upgrade to NAT v1.3

Cite

Please consider citing [our paper](#) when using this code in a project. You can use the citation BibTeX:

```
@inproceedings{zemicheal2024llm,
  title={LLM agents for vulnerability identification and verification of CVEs},
  author={ZeMicheal, Tadesse and Chen, Hsin and Davis, Shawn and Allen, Rachel and Demoret, Michael and Song, Ashi},
  booktitle={Proceedings of the Conference on Applied Machine Learning in Information Security (CAMLIS 2024)},
  pages={161--173},
  year={2024},
  publisher={CEUR Workshop Proceedings},
  volume={3920},
  url={https://ceur-ws.org/Vol-3920/}
}
```

License

By using this software or microservice, you are agreeing to the [terms and conditions](#) of the license and acceptable use policy.

Terms of Use

GOVERNING TERMS: The NIM container is governed by the [NVIDIA Software License Agreement](#) and [Product-Specific Terms for AI Products](#); and use of this model is governed by the [NVIDIA AI Foundation Models Community License Agreement](#).

ADDITIONAL Terms: [Meta Llama 3.1 Community License](#), Built with Meta Llama 3.1.

About

Rapidly identify and mitigate container security vulnerabilities with generative AI.

- [Readme](#)
- [Apache-2.0 license](#)
- [Security policy](#)
- Activity
- Custom properties
- 199** stars
- 13** watching
- 81** forks

[Report repository](#)

Releases

2.1.0 Latest

on Dec 2, 2025

[+ 3 releases](#)

Packages

No packages published

Contributors



[+ 6 contributors](#)

Languages

